

What Does “N× Fewer Tokens” Measure? Baselines, Sampling, and Output Form in Repository-Navigation Benchmarks

Anonymous Author(s)
Anonymized for review

Abstract—Repository-graph tools for coding agents are promoted with large token-reduction ratios. We ask how much such a ratio depends on how it is measured. We contribute NAVBENCH, a fully automatic, model-free benchmark with three label layers that never treat an evaluated tool as ground truth: seeded fixtures whose call sites are known by construction, 17 public Python and TypeScript repositories with independently sampled query targets, and calls observed while running the Python repositories’ test suites. We apply it to two Tree-sitter code-graph tools (one in two versions), ripgrep, and two language servers. Re-applying our own prototype’s earlier benchmark definition to public code reproduces its headline: graph answers are 85× smaller than the whole files they replace. The same graph outputs are only 3.0× smaller than `rg -w` output, and 1.1–2.2× larger than language-server results printed one location per line. Sampling targets from that prototype’s own graph edges raises its answer rate from 44–54% to 99–100%; after adjusting for call fan-out and declaration kind, +32 to +42 points remain, whereas for the other graph tool most of the smaller gap is explained by target composition. On held-out fixture targets that both answer completely, the most complete graph tool returned as many tokens as ripgrep (ratio 1.03, 95% CI 0.94–1.13) but far fewer wrong candidates (caller precision 0.98 vs. 0.35): its advantage was precision, not payload. We release the harness and data, and recommend that efficiency claims report baseline, sampling frame, output form, completeness, and precision together.

I. INTRODUCTION

Foundation-model coding agents spend much of their context budget locating code [9], and graph-based navigation tools, typically exposed to the model as MCP tools, promise to answer structural questions such as “who calls *X*?” from a pre-built index [1], [2], [3], [4]. Such tools are often promoted with a token ratio. A ratio, however, has three hidden parameters that are not properties of the tool: the *baseline* (whole files, grep, or a language server), the *query sample* (often symbols the graph already connects), and the *output form* (names only, locations, or source). Every token such a tool returns is placed in the model’s context, so measured payload directly shapes the efficiency claims made for agent tooling.

Our starting point was a research prototype developed by the authors, G1, whose own benchmark reported 85.8× fewer tokens than reading the referencing files, measured on graph-selected symbols. We include G1 as one of the evaluated tools and report its shortcomings.

We ask: **Q1** How does the baseline change measured payload ratios? **Q2** How does sampling targets independently of the graph change measured coverage and rankings? **Q3**

Under the same output form, and with labels no evaluated tool produced, which tools reach useful token–quality operating points?

Contributions: (1) NAVBENCH, an open benchmark harness with three independent label layers, automatic validity gates, and output-form control; (2) evidence that, for identical graph outputs, the measured ratio spans more than two orders of magnitude depending on baseline and output form; (3) evidence that sampling targets from a tool’s own edges overstates that tool’s coverage beyond what target composition explains; (4) token–recall–precision profiles by call pattern, and recall on runtime-observed calls, for graph, lexical, and language-server retrieval.

II. BENCHMARK DESIGN

Tasks. T1 (primary): given a target declaration, return its direct call sites. The gold unit is a call expression whose callee statically binds to the target. T2: given a call site, return the declaration.

Layer A: seeded fixtures. A generator emits small, executable Python and TypeScript repositories. Each contains one instance of every pattern: direct calls; calls through an import alias, a module or namespace attribute, a re-export, or a typed or `super` receiver; constructor calls; module-level calls; a same-named method on another class; local shadowing; callbacks; string-dispatched calls; decorators or arrow functions; and comment and string distractors. Half of the fixtures use common, reused names (“ambiguous”). The generator writes a manifest of every expected declaration and site. Python fixtures execute, and TypeScript fixtures compile under `-strict` and run. For every Python target, the call lines observed at runtime equal the manifest’s sites. This gate caught three manifest omissions before any held-out data existed. We use 4+4 calibration and 16+16 held-out fixtures: 224 targets, 160 with explicit call sites.

Layer B: natural repositories. We use 9 Python and 8 TS/JS libraries, selected by fixed criteria before results were observed, plus 3 pilot repositories excluded from all results. Declarations are enumerated by tools that share no code with any graph: Python `ast` and the TypeScript compiler API. *S-ind*, the primary sample, is drawn from a random pool of ≤ 300 non-test declarations, stratified by kind and by the language server’s call fan-out, with inclusion weights recorded; it has 661 targets. *S-cond*, a diagnostic sample of 331 targets, is

drawn from declarations with ≥ 1 incoming G1 edge, which is the original benchmark’s rule; 76 declarations fall in both samples. Layer B supports agreement, availability, size, and failure rates, but not accuracy.

Layer C: runtime-observed calls. Each Python repository’s full test suite runs under `sys.monitoring`. A call counts only when the callee’s entry is confirmed, since `CALL` fires before invocation. Calls are mapped to the independent index. We report recall on observed calls, a different population from all calls; unobserved static results are not false positives.

Arms. `rg-w` with 0 and 3 context lines (bare name); `pyright 1.1.414` and the `TypeScript 5.9.3` language service (references at the declaration; its native form is the `LSP Location JSON` rebuilt from the returned URIs and ranges); `G1 find_references`, and `G1’s` original `lookup_symbol+get_dependencies` pair; and `codebase-memory` [4] at `v0.5.5`, the release its paper evaluates, at `v0.5.7`, and at current main (commit `bf93f0b7`). `v0.5.5` and `v0.5.7` gave identical answers on every fixture query; on natural code they differed on 5.8% of queries, no more than `v0.5.7` differs from its own re-run (5.3%), so they share one row (“`v0.5.5/7`”). The `codebase-memory` versions are queried through persistent MCP sessions; when a tool reports ambiguity, the follow-up call uses the target’s qualified name and both calls are counted.

Scoring and statistics. Returned facts are mapped to call sites (location tools only), enclosing caller declarations, and files. A returned location credits its enclosing caller; under a strict rule crediting only locations that are themselves gold call sites, recall of `rg-w` and the language servers changed by ≤ 0.01 and no contrast changed. An answer is *complete* when caller-level recall is 1. Tokens (`cl100k`) are counted on each tool’s native output, a common one-location-per-line form, and a source-enriched form. An automatic check confirms identical fact sets across forms for every query. Estimates are repository-macro means with 95% hierarchical-bootstrap intervals over repositories, then targets. The primary contrast and a $1.5\times$ margin were frozen before held-out data. Post-freeze changes are reported in §V.

III. RESULTS

A. Q1: the baseline determines the ratio

Fig. 1 and Table I give the ratios. Applying `G1’s` original definition to the public repositories reproduces its headline: $85.3\times$ as a repository geometric mean (95% CI 57.2–126.9; $152.8\times$ pooled; 328 targets), consistent with the $85.8\times$ originally reported. That definition uses graph-conditioned targets, whole files selected by the graph, and the `lookup+deps` output. This reproduces `G1’s` own earlier measurement; it neither reproduces nor tests `codebase-memory’s` agent-level $10\times$ result, which measures a different workflow. Against `rg-w`, the *same* outputs are $3.0\times$ smaller (2.4–4.0). Output form alone can flip the sign: the language server’s results are larger than every graph tool’s output as raw JSON, but smaller in the location form (ratios 0.45–0.88). In an exploratory

TABLE I
Q1: BASELINE/GRAPH PAYLOAD RATIO (REPO-MACRO GEOMETRIC MEAN, S-IND).

Graph tool	whole files	<code>rg-w</code>	LSP JSON	LSP loc.
<code>G1 find_references</code>	139.3	6.01	3.10	0.88
<code>G1 lookup+deps</code>	85.6	3.27	2.46	0.54
<code>codebase-memory current</code>	48.0	2.92	1.92	0.45
<code>codebase-memory v0.5.5/7</code>	60.8	3.54	1.46	0.46

TABLE II
Q2: S-COND $\rightarrow$ S-IND. NON-EMPTY ANSWERS (%), THE S-COND–S-IND DIFFERENCE AFTER POST-STRATIFICATION [95% CI], AND CALLER RECALL ON RUNTIME-OBSERVED CALLS (PYTHON).

Arm	non-empty %	adjusted Δ (pts)	obs.-call recall
<code>G1 find_references</code>	100 $\rightarrow$ 54	+32 [21, 41]	0.85 $\rightarrow$ 0.70
<code>G1 lookup+deps</code>	99 $\rightarrow$ 44	+42 [29, 53]	0.81 $\rightarrow$ 0.63
<code>codebase-memory current</code>	88 $\rightarrow$ 60	+9 [2, 17]	0.79 $\rightarrow$ 0.66
<code>codebase-memory v0.5.5/7</code>	84 $\rightarrow$ 62	+8 [0, 15]	0.72 $\rightarrow$ 0.60
language server	97 $\rightarrow$ 83	−1 [−5, 3]	0.84 $\rightarrow$ 0.77
<code>rg-w</code>	100 $\rightarrow$ 100	0	0.97 $\rightarrow$ 0.95

regression, the `rg-w/graph` ratio rises with the number of occurrences of the target’s name in the repository, with an elasticity of ≈ 0.77 (95% CI 0.69–0.87). Lexical retrieval is costly mainly for common, ambiguous names.

B. Q2: graph-conditioned sampling overstates coverage

Table II compares the two samples. S-cond is conditioned on `G1’s` edges, which also changes target composition: 9% of S-cond but 35% of S-ind targets have no call site according to the language server, and declaration kinds and name frequencies differ. For those targets an empty answer is correct. We therefore post-stratify S-cond to S-ind’s distribution over repository $\times$ kind $\times$ fan-out bucket (0, 1–2, 3–9, ≥ 10 call sites). The adjustment removes composition effects: the language server, on which S-cond is not conditioned, shows no adjusted difference. For `G1`, the conditioning tool, the answer-rate gap remains large (+32 and +42 points), while its observed-call recall gap shrinks to +0.13 (95% CIs reaching 0). For `codebase-memory` the adjusted gaps are small (+8 to +9 points; recall +0.08–0.09, CIs including 0); conditioning inflates mainly the coverage of the tool whose edges define the sample. Independently sampled declarations are often absent from `G1`: 14% of S-ind targets are missing. `G1’s` coverage of non-test declarations is 91–100% in the Python repositories, but 47–68% in five of the eight TS/JS repositories. The causes are unindexed object-literal arrow functions, and same-named nested declarations that collapse into one node (e.g., 303 test-local classes in one repository).

C. Q3: correctness and tokens at matched quality

Table III and Fig. 2 report the held-out fixtures. For the primary contrast we compute the geometric-mean token ratio of each arm to `rg-w` over the targets on which *both* are complete. These subsets differ by arm (n of 160 below); they are conditioned on success and are not globally matched operating points. A saving counts as meaningful only if the whole interval lies below $1/1.5 \approx 0.667$.

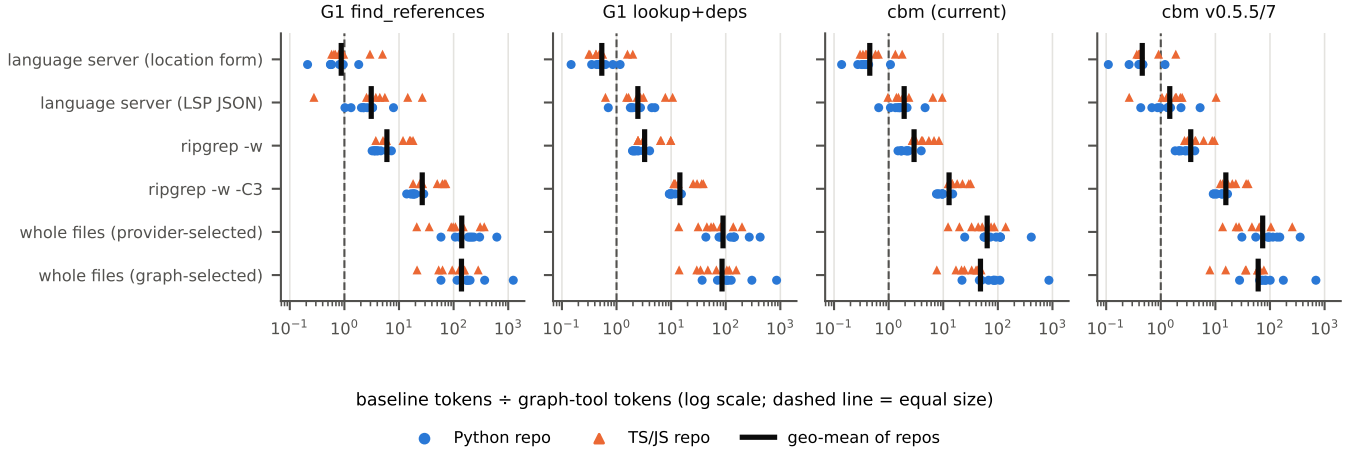


Fig. 1. Q1. Payload of each baseline relative to each graph tool’s native output, for the same queries (S-ind, non-empty graph answers). Points are repository geometric means; bars are the mean across repositories. The ratio moves by over two orders of magnitude with the choice of baseline and output form.

- G1 find_references: 0.35 [0.33, 0.37] (n=64), a saving, but complete on only 40% of targets, against 80% for rg -w;
- G1 lookup+deps: 0.65 [0.62, 0.672] (n=64), a 1.5× point saving whose interval crosses the threshold, so not meaningful;
- codebase-memory current: 1.03 [0.94, 1.13] (n=127), no evidence of a substantial saving (this is not an equivalence test); it is the only graph tool as complete as the baselines (89%);
- codebase-memory v0.5.5/7: 0.58 [0.56, 0.62] (n=79), a saving at 49% completeness;
- language-server JSON: 1.75 [1.65, 1.85] (n=128), a cost.

Over all 160 targets, without conditioning on completeness, codebase-memory current’s ratio is 1.01 [0.93, 1.09], and the less complete arms’ ratios fall to 0.33–0.63. Small outputs coincided with incomplete answers.

Matched recall is not matched quality. codebase-memory current and rg -w have the same recall and similar tokens, but codebase-memory returns almost no wrong callers (precision 0.98 vs. 0.35; F1 0.95 vs. 0.50). The fixtures plant same-named methods, shadowing locals, and comment and string mentions, so they are deliberately hard for lexical search; we do not measure precision on natural code. Output form also matters here: in the common one-fact-per-line form, every graph tool needs only 0.24–0.28 of rg -w’s tokens on the same jointly complete targets. The forms differ in content, though: graph lines name callers, while rg -w lines list every occurrence of the name. codebase-memory’s native parity with rg -w comes from its verbose tree output (160 native vs. 19 location-form tokens).

Failures by pattern are as follows. rg -w misses only calls through import aliases. pyright also misses them; the TypeScript service does not. Of G1’s 240 missed callers (lookup+deps): 96 are wrong resolutions to a same-named local function that shadows the target; 32 are module-level

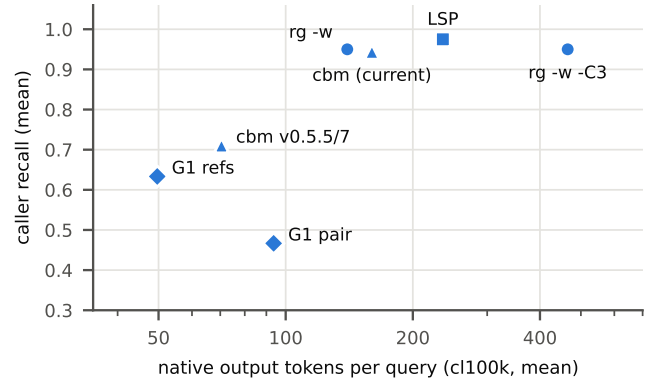


Fig. 2. Q3. Held-out fixtures, Python and TypeScript pooled: native output tokens vs. caller-level recall per arm (repository means; G1 pair = lookup+deps). Small outputs coincide with low recall; the only graph tool with baseline-level recall needs baseline-level tokens.

calls, which it never attributes to a caller; and 112 are relations it does not record (re-exports, TS constructor calls, and typed-receiver or super calls). Ambiguous names lower G1’s recall from 0.73 to 0.53, but leave codebase-memory current unaffected (0.95 vs. 0.94). codebase-memory v0.5.5/7 misses the typed-receiver and constructor calls that the current version finds. In T2 (definition lookup), every arm found every definition. Only the language server returned exactly one answer (precision 1.00); the other arms returned same-named candidates (precision 0.68–0.76), at 37–98 tokens.

D. Layer C: recall on observed calls

On 323 Python targets with observed calls, caller recall was: rg -w 0.95 [0.92, 0.98]; language server 0.79 [0.64, 0.90]; G1 0.75 and 0.69; codebase-memory 0.70 (current) and 0.64 (v0.5.5/7). The language server’s misses concentrate in repositories that ship .pyi stubs, because pyright binds public-API calls to the stub declarations rather than the implementation. The graph tools’ 0.64–0.75 recall is in the range PyCG reports

TABLE III
Q3: HELD-OUT FIXTURES, T1, CALLER LEVEL, 160 TARGETS. TOKENS:
MEAN NATIVE, AND MEAN IN THE COMMON LOCATION FORM.

Arm	recall	precision	complete %	native	loc.
rg-w	0.95	0.35	80.0	140	65
rg-w -C3	0.95	0.35	80.0	465	65
language server	0.97	0.64	90.0	236	39
G1 find_references	0.63	0.68	40.0	50	18
G1 lookup+deps	0.47	1.00	40.0	94	6
codebase-memory current	0.94	0.98	89.4	160	19
codebase-memory v0.5.5/7	0.71	0.90	49.4	70	15

for static Python call graphs (≈ 0.70 [8]). Their position below lexical search is consistent with Codebase-Memory’s agent-level comparison against a grep-and-read explorer (83% vs. 92% answer quality [4]).

IV. RECOMMENDATIONS

- Report the baseline, and more than one: the same graph outputs measured $0.45\times$ to $141\times$.
- Sample targets independently of the tool under test: sampling from G1’s own edges raised its answer rate by 32–42 points even after adjusting for target composition.
- Count tokens in a common output form, and state what each form contains: names-only outputs carry less information.
- Report completeness and precision next to tokens: on jointly complete targets, the most complete graph tool saved no tokens over rg-w, but was far more precise.
- Do not equate payload tokens with agent cost. Agent behaviour and prompt caching are out of scope here, and agent-level studies [4], [5] measure something different.

V. THREATS TO VALIDITY

Construct. Payload tokens are not agent cost [9], [10]. Static possible-target labels encode one semantics. Caller-level scoring credits any location inside a true caller; a strict variant changes no result.

Internal. G1 was developed by the authors; to limit bias, every tool is queried through its own interface under one frozen configuration, and G1’s failures are reported in full. After freezing, we found that pyright answers before its workspace is indexed. We added a readiness wait (0/80 reference sets changed on re-query 30 s later) and re-ran all Python jobs. This raised layer-C language-server recall from 0.73 to 0.79 and changed no conclusion; pre-fix results are released. Also after freezing, and in response to review: we fixed a scoring error that had dropped the 76 S-ind targets also drawn into S-cond (S-ind numbers changed, no conclusion did; both versions are released); we added codebase-memory v0.5.5 on the frozen targets with the v0.5.7 adapter; and we added four post-hoc analyses (full-workload ratios, output-form contrasts, strict caller credit, post-stratified sampling), labelled as such in the released reports. codebase-memory v0.5.x answers for same-named declarations vary between runs (5% of natural queries; current version 0.4%). Before freezing, name-resolution rules were corrected so that tool outputs are not penalised by our

own matching. Unresolved output (0–6%) still counts against the tool. Tokens were counted with gpt-tokenizer’s bundled cl100k ranks, validated against known token ids.

External. Fixtures are cleaner than real code. The repositories are a purposive selection, and only two languages are covered. Layer C covers only exercised code. There is no human audit, so accuracy on natural code is not measured.

VI. RELATED WORK

Code graphs for agents improve repository-level tasks [1], [2], [3]. Codebase-Memory [4] reports about $10\times$ fewer agent tokens than an agent using file reading and grep, at somewhat lower answer quality (83% vs. 92%). Xu [5] compares semantic and lexical retrieval inside an agent, including task success, output enrichment, and when token savings arise; we instead measure tools without a model, across graph, lexical, and language-server retrieval, against labels no evaluated tool produced, and quantify sampling effects. CodeNib [6] compares static and live navigation on graph-derived queries. For agent memory over long conversations, not code, grep often beats vector retrieval [7]. PyCG bounds what static call graphs achieve [8]. We complement these agent-level studies with a model-free, cross-tool measurement.

VII. DATA AVAILABILITY

Harness, fixtures generator, pinned corpus commits, raw per-query outputs, traces, analysis, and the frozen protocol: <https://anonymous.4open.science/r/XXXX> (anonymized).

REFERENCES

- [1] S. Ouyang *et al.*, “RepoGraph: Enhancing AI software engineering with repository-level code graph,” in *Proc. ICLR*, 2025.
- [2] X. Liu *et al.*, “CodexGraph: Bridging large language models and code repositories via code graph databases,” in *Proc. NAACL*, 2025.
- [3] Z. Chen *et al.*, “LocAgent: Graph-guided LLM agents for code localization,” in *Proc. ACL*, 2025.
- [4] M. Vogel *et al.*, “Codebase-Memory: Tree-sitter-based knowledge graphs for LLM code exploration via MCP,” arXiv:2603.27277, 2026.
- [5] P. Xu, “Does a language server save tokens for coding agents? A measurement methodology and preliminary study,” arXiv:2608.13568, 2026.
- [6] “CodeNib: A multi-view data system for serving repository context to coding agents,” arXiv:2607.25431, 2026.
- [7] S. Sen *et al.*, “Is grep all you need? How agent harnesses reshape agentic search,” arXiv:2605.15184, 2026.
- [8] V. Salis *et al.*, “PyCG: Practical call graph generation in Python,” in *Proc. ICSE*, 2021.
- [9] L. Bai *et al.*, “How do AI agents spend your money? Analyzing and predicting token consumption in agentic coding tasks,” arXiv:2604.22750, 2026.
- [10] S. Weinberger and A. Hozes, “Token reduction is not cost reduction: An empirical study of end-to-end efficiency in API-based coding agents,” arXiv:2607.12161, 2026.